Processamento de transações

Oracle Academy

DP_18_1_pr_Transações de Banco de Dados

Roteiro da aula

- Conceito de transação
- Propriedades ACID
- Estrutura de uma transação
- Comandos para controle de transação
- Exercício dirigido
- Exercícios avaliativos

O que é uma transação?

- Uma transação é um programa em execução que forma uma unidade lógica de processamento de banco de dados
- Inclui uma ou mais operações de acesso ao banco de dados
  - inserção, exclusão, modificação ou recuperação
- Utilizada para agrupar operações logicamente relacionadas e assegurar a consistência

Exemplo: sistema bancário

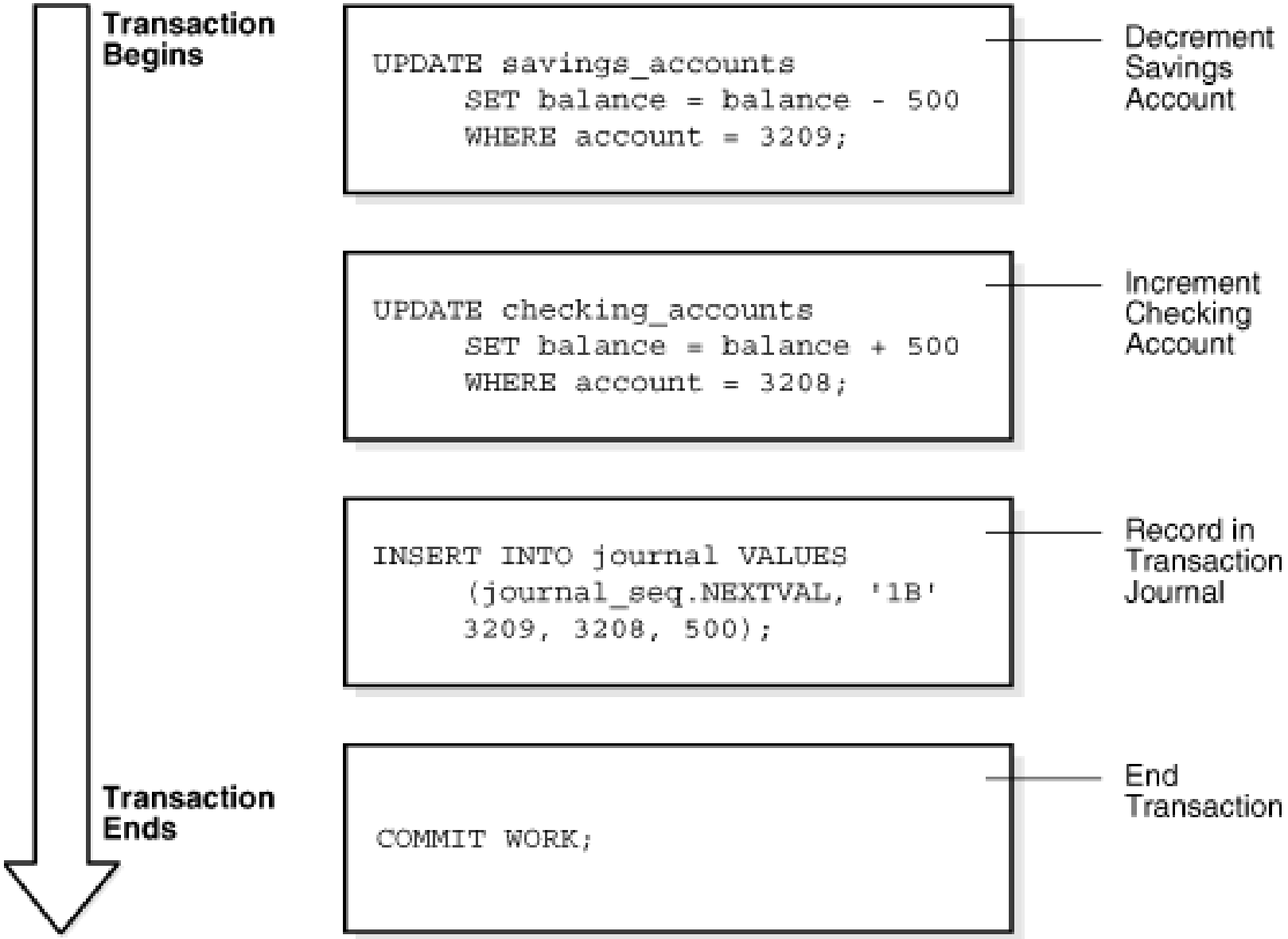
Quando um cliente transfere $ da conta poupança para a conta corrente, a transação deve conter 3 operações:

- Decrementar o valor da conta poupança
- Incrementar o valor na conta corrente
- Registrar a transação numa tabela de auditoria (journal)

Exemplo: sistema bancário

- Para manter a consistência, as 3 operações devem ocorrer de forma atômica
- Se acontecer um problema como
  - saldo insuficiente
  - número de conta inválido ou
  - uma falha de hardware
- Então o SGBD deve desfazer a transação por inteiro de modo que o saldo de todas as contas seja correto

Exemplo: sistema bancário



Importância das transações

- Controle de concorrência
  - Transações submetidas por diversos usuários podem interferir uma nas outras e produzir resultados incorretos

- Recuperação em caso de falhas
  - Transações interrompidas por falhas no sistema devem ser tratadas para evitar erros ou inconsistências

Propriedades ACID

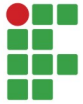
- Atomicidade
  - Uma transação é uma unidade de processamento atômica – deve ser executada em sua totalidade ou não ser realizada de forma alguma

- Preservação da Consistência
  - Uma transação deve levar o BD de um estado consistente para outro

Propriedades ACID

- Isolamento
  - A execução de uma transação não deve ser interferida por quaisquer outras transações que acontecem simultaneamente
  - O efeito de uma transação não é visível para outras transações até que a transação seja confirmada (COMMIT)

- Durabilidade ou permanência
  - As alterações aplicadas por uma transação confirmada devem persistir no BD



Estrutura de uma transação

- Início de uma transação
  - Primeiro comando DML executado

- Fim de uma transação
  - Comandos COMMIT ou ROLLBACK
  - Um comando DDL ou DCL (COMMIT implícito)
  - Fim da conexão com o SGBD
  - Falha do sistema

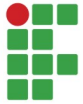
Comandos para transação em SQL

- Para tornar um conjunto de instruções SQL atômico → usar comandos para transações
  - COMMIT
  - ROLLBACK
  - SAVEPOINT
  - ROLLBACK TO SAVEPOINT

Controle de transação

- COMMIT
  - Finaliza a transação corrente e torna permanentes todas as alterações realizadas na transação

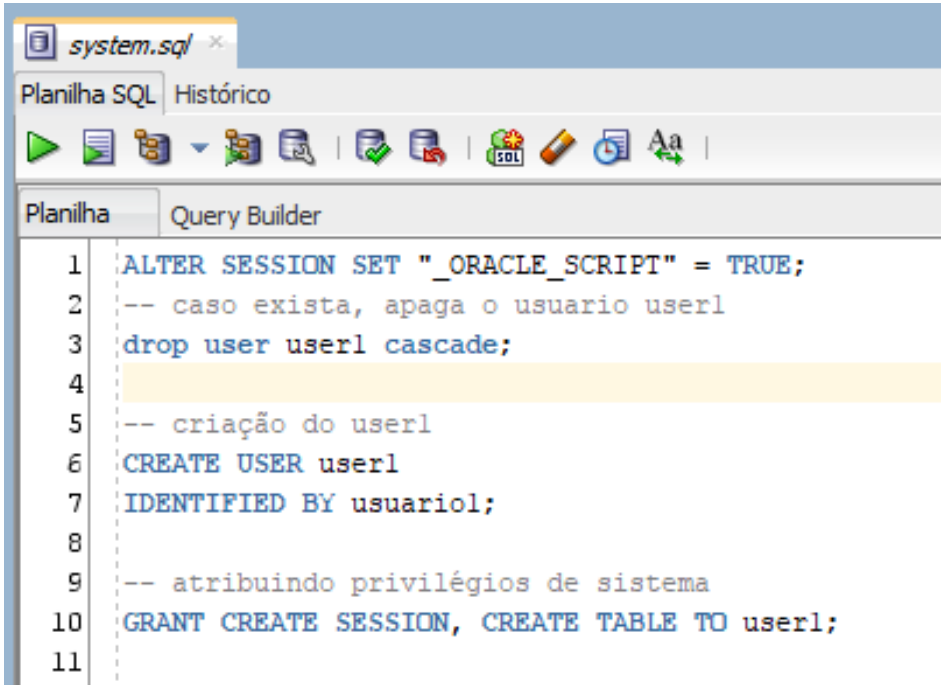
- ROLLBACK
  - Desfaz as alterações realizadas na transação atual – faz com que todas as alterações realizadas desde o início da transação sejam descartadas

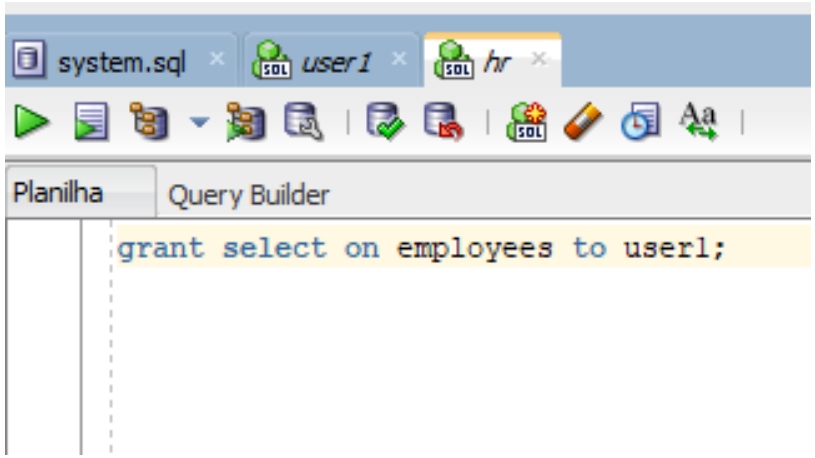


Prática

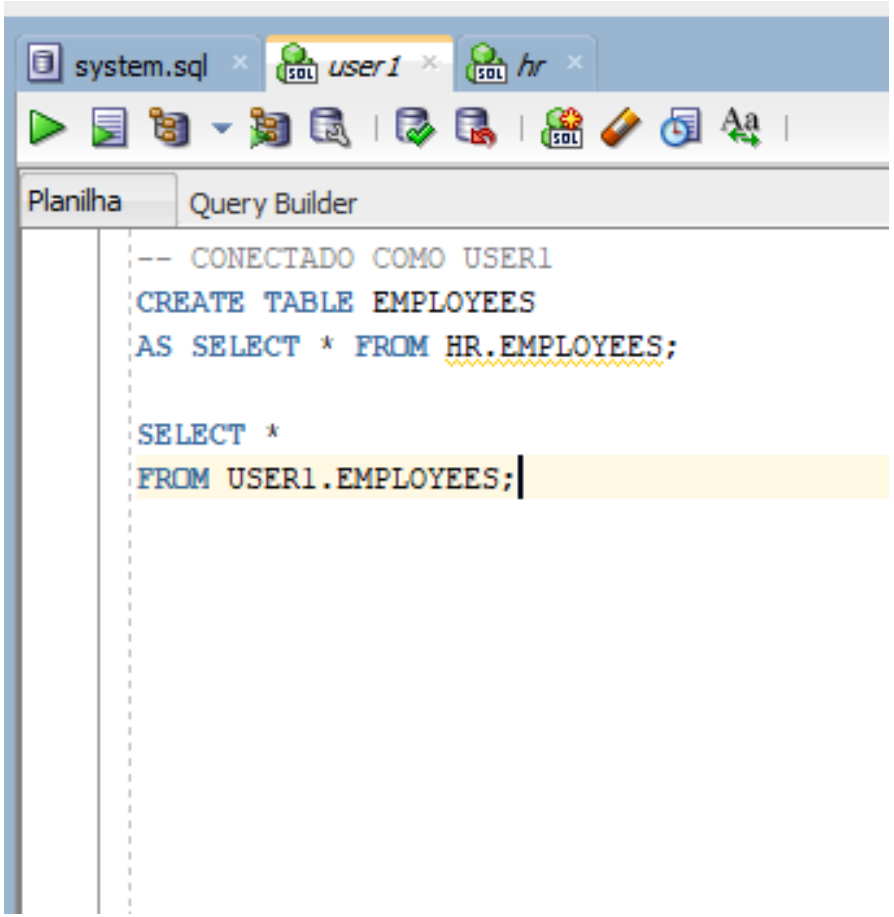
- Criar usuário ‘USER1’ que deve ter uma tabela employees contendo uma cópia do conteúdo da tabela hr.employees
  - Escreva os comandos SQL necessários

Preparação


system.sql
Planilha SQL
Histórico
Planilha
Query Builder
1 ALTER SESSION SET "_ORACLE_SCRIPT" = TRUE;
2 -- caso exista, apaga o usuario user1
3 drop user user1 cascade;
4
5 -- criação do user1
6 CREATE USER user1
7 IDENTIFIED BY usuario1;
8
9 -- atribuindo privilégios de sistema
10 GRANT CREATE SESSION, CREATE TABLE TO user1;
11


system.sql
user1
hr
Planilha
Query Builder
grant select on employees to user1;

usuário ‘USER1’ deve ter uma tabela employees contendo uma cópia do conteúdo da tabela hr.employees



Estrutura de uma transação

```
-- executar como usuário SYSTEM
SELECT XID, STATUS, START_TIME FROM V$TRANSACTION;

-- executar como usuário user1
SELECT * FROM EMPLOYEES WHERE EMPLOYEE_ID IN (100, 110);

UPDATE EMPLOYEES  -- início da transação
SET SALARY = SALARY - 1000
WHERE EMPLOYEE_ID = 100;

UPDATE EMPLOYEES
SET SALARY = SALARY + 1000
WHERE EMPLOYEE_ID = 110;

COMMIT; -- fim da transacao
```

Estrutura de uma transação

```
-- executar como usuário USER1
UPDATE employees -- início da transação
SET salary = 20000;

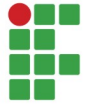
-- executar como usuário SYSTEM
SELECT XID, STATUS, START_TIME FROM V$TRANSACTION;


-- executar como usuário USER1
ROLLBACK; -- fim da transação
```

Controle de transação

- SAVEPOINT
  - Identifica um ponto marcador em uma transação para o qual é possível realizar ROLLBACK

- ROLLBACK TO SAVEPOINT <nome_savepoint>
  - desfaz as alterações realizadas a partir do SAVEPOINT <nome_savepoint>
  - mas não desfaz a transação inteira



Controle de transação

```
UPDATE employees SET salary = 7000
    WHERE last_name = 'Banda';

SAVEPOINT after_banda_salary;

UPDATE employees SET salary = 12000
    WHERE last_name = 'Greene';

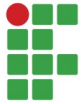
SAVEPOINT after_greene_salary;

ROLLBACK TO SAVEPOINT after_banda_salary;

COMMIT;
```

Transações ativas

- Transação ativa: iniciada mas ainda não confirmada (COMMIT) nem cancelada (ROLLBACK)
  - As mudanças feitas são temporárias
- Antes da transação terminar, o estado dos dados é:
  - SGBD mantém os valores antigos dos dados modificados pela transação (tablespace UNDO)
  - As modificações são armazenadas em buffers para serem escritas definitivamente no BD se houver COMMIT
  - As linhas afetadas pelas modificações são bloqueadas. Outros usuários não podem alterar os dados nas linhas afetadas nem podem visualizar as mudanças não confirmadas



Transações ativas

- Enquanto uma transação não termina, as linhas afetadas ficam bloqueadas (LOCK)

- Outros usuários não podem alterar dados nas linhas bloqueadas, até que a transação que as bloqueou seja terminada
  - Mas podem fazer consultas SELECT obtendo os valores anteriores à transação ativa

Prática 2 (preparação)

Criar dois usuários: USER2 e USER3, ambos com privilégios para consultar e modificar os dados de USER1.EMPLOYEES

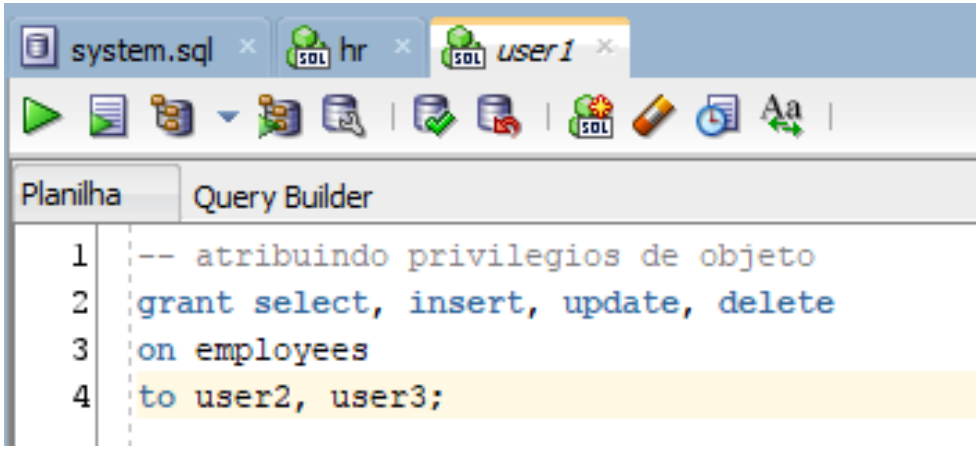
- Escreva os comandos SQL necessários

Prática 2

Conectado como system

```
11
12  -- prática 2
13  drop user user2 cascade;
14  drop user user3 cascade;
15
16  -- criação do user2
17  CREATE USER user2
18  IDENTIFIED BY usuario2;
19
20  -- criação do user3
21  CREATE USER user3
22  IDENTIFIED BY usuario3;
23
24  -- atribuindo privilégios de sistema
25  GRANT CREATE SESSION TO user2, user3;
26
27
```

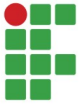
Conectado como user1



Regras para bloqueios

Uma linha somente é bloqueada quando modificada por alguma transação

- Quando um comando atualiza um linha, a transação adquire o LOCK somente para esta linha
- O bloqueio em nível de linha minimiza contenção de dados



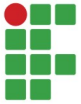
Exemplo

T	USER1	USER2
t0	UPDATE employees SET SALARY = SALARY + 1000 WHERE EMPLOYEE_ID = 100;	
t1		UPDATE user1.employees SET SALARY = SALARY + 1000 WHERE EMPLOYEE_ID = 101;
t2	COMMIT;	
t3		COMMIT;

Regras para bloqueios

Uma transação de escrita em uma linha bloqueia outra transação de escrita concorrente na mesma linha.

- Se uma transação está modificando uma linha, então o bloqueio evita que uma transação diferente modifique a mesma linha simultaneamente.



Exemplo

T	USER1	USER2
t0	UPDATE employees SET SALARY = SALARY + 1000 WHERE EMPLOYEE_ID = 100;	
t1		UPDATE user1.employees SET SALARY = SALARY + 2000 WHERE EMPLOYEE_ID = 100;
t2	COMMIT;	
t3		COMMIT;

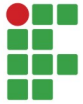
Regras para bloqueios

Uma transação de escrita nunca bloqueia um comando de leitura

- Quando uma linha está sendo modificada por uma transação, o SGBD permite que outras transações a consultem, considerando valores anteriores ao início da transação de escrita
- Quando é feito o COMMIT as mudanças são visíveis para outras transações de consulta

Exemplo

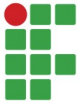
T	USER1	USER2
t0	SELECT EMPLOYEE_ID, SALARY FROM EMPLOYEES WHERE EMPLOYEE_ID = 100;	
t1	UPDATE employees SET SALARY = SALARY + 1000 WHERE EMPLOYEE_ID = 100;	
t2		SELECT * FROM user1.employees WHERE EMPLOYEE_ID = 100;
t3	COMMIT;	
t4		SELECT * FROM user1.employees WHERE EMPLOYEE_ID = 100;



Regras para bloqueios

Um comando de leitura nunca bloqueia uma transação de escrita

- Como um comando de leitura de uma linha não a bloqueia, uma transação de escrita pode modificar essa linha
- A única exceção é o comando SELECT ... FOR UPDATE, que bloqueia as linhas sendo consultadas



Exemplo

T	USER1	USER2
t0	SELECT EMPLOYEE_ID, SALARY FROM EMPLOYEES WHERE EMPLOYEE_ID = 100;	
t1		UPDATE user1.employees SET SALARY = SALARY + 2000 WHERE EMPLOYEE_ID = 100;
t2	SELECT EMPLOYEE_ID, SALARY FROM EMPLOYEES WHERE EMPLOYEE_ID = 100;	
t3		COMMIT;
t4	SELECT * FROM EMPLOYEES WHERE EMPLOYEE_ID = 100;	SELECT * FROM user1.employees WHERE EMPLOYEE_ID = 100;

SELECT FOR UPDATE

Comando SELECT FOR UPDATE bloqueia as linhas retornadas pela consulta.

```
SELECT *
FROM EMPLOYEES
WHERE job_id = 'SA_REP'
FOR UPDATE;

COMMIT; -- libera o bloqueio
```

Geralmente usado para evitar que outros usuários/sessões modifiquem os valores durante ou logo após a consulta.

Exemplo

T	USER1	USER2
t0	SELECT EMPLOYEE_ID, SALARY FROM EMPLOYEES WHERE EMPLOYEE_ID = 100 FOR UPDATE;	
t1		UPDATE user1.employees SET SALARY = SALARY + 2000 WHERE EMPLOYEE_ID = 100;
t2	UPDATE employees SET SALARY = SALARY + 1000 WHERE EMPLOYEE_ID = 100;	
t3	COMMIT;	
t4		COMMIT;
t5	SELECT * FROM EMPLOYEES WHERE EMPLOYEE_ID = 100;	SELECT * FROM user1.employees WHERE EMPLOYEE_ID = 100;

LOCKs e DEADLOCKs

Oracle automaticamente detecta DEADLOCKs e os resolve ao desfazer um dos comandos que gerou o DEADLOCK, liberando a outra transação que estava bloqueada.

T	USER1	USER2
t0	UPDATE employees SET salary = salary*1.1 WHERE employee_id = 100;	UPDATE USER1.employees SET salary = salary*1.1 WHERE employee_id = 200;
t1	UPDATE employees SET salary = salary*1.1 WHERE employee_id = 200;	UPDATE USER1.employees SET salary = salary*1.1 WHERE employee_id = 100;

Exercício dirigido

Considerando os comandos para controle de transação, regras para bloqueio e tipos de bloqueio, analise os comandos ilustrados na tabela (próximo slide) e responda:

1. Alguma transação tem que aguardar o término de outra transação em algum momento? Explique sua resposta.
2. Qual o resultado das consultas realizadas em t3 para cada sessão? Explique sua resposta.

T	Sessão 1	Sessão 2	Sessão 3
t0	SELECT employee_id, salary FROM employees WHERE employee_id IN (100, 101);	SELECT employee_id, salary FROM employees WHERE employee_id IN (100, 101);	SELECT employee_id, salary FROM employees WHERE employee_id IN (100, 101);
t1	UPDATE employees SET salary=salary+100 WHERE employee_id=100;		
t2		UPDATE employees SET salary=salary+100 WHERE employee_id=101;	
t3	SELECT employee_id, salary FROM employees WHERE employee_id IN (100, 101);	SELECT employee_id, salary FROM employees WHERE employee_id IN (100, 101);	SELECT employee_id, salary FROM employees WHERE employee_id IN (100, 101);

Importante: use conexões com usuários diferentes ou abra Planilhas Não Compartilhadas para as sessões. Por padrão, abrir várias planilhas em uma mesma conexão reutiliza o canal e não cria uma nova sessão no banco de dados